

Introduction and Applications of Quantum Models

QML

Contents

1	Overview	3
2	Taxonomy of QML Models	3
2.1	The Three Paradigms	3
2.2	Comparison Table	4
3	Quantum Models for Classification	4
3.1	Variational Quantum Classifier (VQC)	4
3.2	Quantum Support Vector Classifier (QSVC)	5
3.3	Comparison of Classification Approaches	5
4	Quantum Models for Regression	5
4.1	Variational Quantum Regressor (VQR)	5
4.2	Quantum Kernel Ridge Regression (QKRR)	6
4.3	Multi-Output Regression	6
5	Quantum Models for Generative Tasks	7
5.1	Born Machines	7
5.2	Quantum GANs (qGANs)	7
6	Quantum Models for Reinforcement Learning	8
6.1	The Setup	8
6.2	Quantum Policy Gradient	8
7	Data Reuploading	9
7.1	The Core Idea	9
7.2	Data Reuploading Circuit	9
7.3	Why It Works: Fourier Analysis	9
7.4	Practical Considerations	10
8	Quantum Transfer Learning	10
8.1	The Architecture	10
8.2	Architecture Diagram	11

8.3	Why This Makes Sense	11
9	Hybrid Classical-Quantum Architectures	11
9.1	Architecture Patterns	11
9.2	The Dressed Quantum Circuit	12
9.3	Practical Hybrid Training	12
10	Real-World Application Areas	12
10.1	Finance	13
10.2	Chemistry and Drug Discovery	13
10.3	Optimization	14
10.4	Materials Science	14
11	Honest Assessment: What Actually Works	14
11.1	What Works (or at Least Shows Promise)	14
11.2	What Doesn't Work (Yet)	15
11.3	What Needs More Qubits	15
12	Looking Forward	15
12.1	Near-Term Opportunities	15
12.2	The Path to Practical Advantage	16
13	Summary	16

1 Overview

weve spent the last several lectures building up the theory — variational circuits, feature maps, kernels, expressibility. now its time to zoom out and ask: wt can u actually do with all this? wt kinds of problems can quantum models solve, and which approaches r best for which tasks?

this lecture is a survey-style overview. well taxonomize quantum ML models, walk through how they apply to classification, regression, generative modeling, and reinforcement learning, then look at some of the most interesting architectural ideas (data reuploading, transfer learning, hybrid systems). well finish with a blunt assessment of where real-world QML applications actually stand.

tbbh this is the lecture where we connect theory to practice. some of it is exciting, some of it is sobering. thats the state of the field right now.

2 Taxonomy of QML Models

before diving into specific applications, lets organize the landscape. there r broadly three paradigms for quantum ML:

2.1 The Three Paradigms

Definition 2.1 (Quantum Kernel Methods). quantum kernel methods use a quantum computer only to evaluate a kernel function $k(x_i, x_j) = |\langle \phi(x_i) | \phi(x_j) \rangle|^2$, then feed the kernel matrix to a classical ML algorithm (SVM, kernel ridge regression, etc.). no variational optimization on the quantum device.

Definition 2.2 (Variational (Hybrid) Methods). variational methods use a parameterized quantum circuit $U(\theta, x)$ as the model, with parameters θ optimized via a classical-quantum feedback loop. the quantum device evaluates the model, the classical computer updates parameters.

Definition 2.3 (Fault-Tolerant Quantum ML). fault-tolerant algorithms assume access to error-corrected qubits and can run deep circuits. examples include quantum PCA (lloyd et al. 2014), quantum sampling (kerenidis-prakash), HHL-based methods. these need thousands to millions of logical qubits.

Intuition. think of it like a spectrum of quantum resource requirements. kernel methods need the least (just state prep + measurement), variational methods need moderate resources (parameterized circuits + optimization), and fault-tolerant methods need the most (deep circuits, error correction). we r currently in the “variational era” bcs thats wt NISQ hardware can handle.

2.2 Comparison Table

Property	Kernel Methods	Variational	Fault-Tolerant
hardware era	NISQ	NISQ	post-NISQ
circuit depth	shallow (fixed)	moderate (optimized)	deep (algorithmic)
classical optimization	SVM/KRR (convex)	gradient descent (non-convex)	minimal
trainability	guaranteed (convex)	barren plateaus risk	N/A (algorithmic)
expressiveness	fixed by kernel	tunable via ansatz	full quantum
data encoding	feature map circuit	embedded in ansatz	amplitude encoding
scaling bottleneck	kernel matrix $O(n^2)$	gradient estimation	qubit count + error correction
provable advantage	yes (liu et al. 2021)	limited results	yes (for specific tasks)

Key Point. no single paradigm dominates. kernel methods have the best theoretical guarantees but scale poorly with dataset size. variational methods r most flexible but face trainability issues (more on this in lecture 16). fault-tolerant methods have the strongest advantages but need hardware we dont have yet.

3 Quantum Models for Classification

classification is the most studied task in QML, and for good reason — its clean, well-defined, and easy to benchmark.

3.1 Variational Quantum Classifier (VQC)

the VQC is the workhorse of near-term quantum classification. recall the structure:

- (i) encode classical data x into quantum state via feature map $S(x)$
- (ii) apply parameterized unitary $W(\theta)$
- (iii) measure expectation value $\langle Z \rangle$ or similar observable
- (iv) threshold to get class label

the prediction function is:

$$f(x; \theta) = \langle 0 | S^\dagger(x) W^\dagger(\theta) M W(\theta) S(x) | 0 \rangle \quad (1)$$

where M is a measurement observable (often Z_0 for binary classification).

training objective — minimize cross-entropy or hinge loss:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(y_i, f(x_i; \theta)) \quad (2)$$

Example 3.1. for binary classification with labels $y \in \{-1, +1\}$ and hinge loss:

$$\ell(y, \hat{y}) = \max(0, 1 - y \cdot \hat{y})$$

this is exactly a quantum version of a soft-margin SVM in the variational picture.

3.2 Quantum Support Vector Classifier (QSVC)

the kernel approach to classification. u compute the quantum kernel matrix:

$$K_{ij} = |\langle 0 | S^\dagger(x_i) S(x_j) | 0 \rangle|^2 \quad (3)$$

then solve the standard SVM dual problem:

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij} \quad (4)$$

subject to $0 \leq \alpha_i \leq C$ and $\sum_i \alpha_i y_i = 0$.

Key Point. the advantage of QSVC over VQC: the optimization is convex. no barren plateaus, no local minima. the disadvantage: u need to compute the full $n \times n$ kernel matrix, which takes $O(n^2)$ quantum circuit evaluations. for large datasets this is a killer.

3.3 Comparison of Classification Approaches

Method	Training	Prediction	Best For
VQC	non-convex opt	1 circuit eval	medium datasets
QSVC	convex (SVM)	$O(n_{sv})$ evals	small datasets
QNN (deep)	non-convex opt	1 circuit eval	complex patterns
projected kernel	convex + classical	classical	large datasets

the projected kernel method (huang et al. 2021) is worth noting: instead of using the fidelity kernel $|\langle \phi(x) | \phi(y) \rangle|^2$, u measure individual qubit expectation values and use those as classical features. this avoids the $O(n^2)$ kernel matrix computation and can be combined with any classical ML method.

$$\text{projected features: } \tilde{\phi}(x) = (\langle Z_1 \rangle_x, \langle Z_2 \rangle_x, \dots, \langle Z_n \rangle_x, \langle Z_1 Z_2 \rangle_x, \dots) \quad (5)$$

4 Quantum Models for Regression

regression is less studied than classification in QML but follows similar principles.

4.1 Variational Quantum Regressor (VQR)

same circuit structure as VQC, but now the output is continuous:

$$f(x; \theta) = \langle 0 | S^\dagger(x) W^\dagger(\theta) M W(\theta) S(x) | 0 \rangle \in [-1, 1] \quad (6)$$

u can rescale to arbitrary range: $\hat{y} = a \cdot f(x; \theta) + b$ where a, b r learnable or fixed.

training uses MSE loss:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i; \theta))^2 \quad (7)$$

Intuition. theres smtg subtle here. bcs the output of a quantum measurement is bounded $[-1, 1]$ (for expectation of pauli observables), VQR naturally produces bounded predictions. this can be a feature (built-in regularization) or a bug (cant predict extreme values) depending on ur task.

4.2 Quantum Kernel Ridge Regression (QKRR)

the kernel approach to regression. given quantum kernel matrix K , solve:

$$\hat{\alpha} = (K + \lambda I)^{-1} y \quad (8)$$

where $\lambda > 0$ is the regularization parameter. prediction for new point:

$$\hat{y}(x_*) = \sum_{i=1}^N \hat{\alpha}_i k(x_i, x_*) \quad (9)$$

this is identical to classical kernel ridge regression but with a quantum kernel. the solution is unique (no local minima), and the regularization λ prevents overfitting.

Theorem 4.1 (Representer Theorem for QKRR). the optimal predictor in the RKHS associated with quantum kernel k lies in the span of the training kernel evaluations:

$$f^*(x) = \sum_{i=1}^N \alpha_i^* k(x_i, x)$$

this holds regardless of the loss function (as long as its monotonically increasing in $\|f\|_{\mathcal{H}}$).

Example 4.1. QKRR for fitting a 1D function. encode $x \in [0, 2\pi]$ with:

$$S(x) = \prod_{j=1}^n R_Z(x) R_Y(x^2) \cdot \text{CNOT layer}$$

the quantum kernel captures nonlinear correlations that would require high-degree polynomials classically. for smooth functions with periodic structure, quantum kernels can be remarkably efficient.

4.3 Multi-Output Regression

for vector-valued regression $y \in \mathbb{R}^m$, u have two options:

1. multiple observables: measure different pauli operators for each output dimension

$$f_k(x; \theta) = \langle \psi(x, \theta) | M_k | \psi(x, \theta) \rangle, \quad k = 1, \dots, m \quad (10)$$

2. multiple circuits: use a separate parameterized circuit for each output (more parameters, more flexible, more expensive)

the first approach is more parameter-efficient but couples all outputs through shared circuit parameters. the second is more flexible but scales linearly in the number of outputs.

5 Quantum Models for Generative Tasks

generative modeling is arguably the most “quantum-native” ML task, bcs quantum mechanics is fundamentally about probability distributions.

5.1 Born Machines

Definition 5.1 (Born Machine). a Born machine is a generative model that directly uses the Born rule to define a probability distribution:

$$p_{\theta}(x) = |\langle x | U(\theta) | 0^n \rangle|^2 \quad (11)$$

where $U(\theta)$ is a parameterized quantum circuit and $|x\rangle$ is a computational basis state.

this is beautiful in its simplicity: the quantum circuit directly is the generative model. sampling is trivial — just run the circuit and measure. the hard part is training.

training approaches:

- (i) MLE: maximize $\sum_i \log p_{\theta}(x_i)$ — but computing $p_{\theta}(x)$ requires exponentially many measurements for general circuits
- (ii) MMD minimization: minimize maximum mean discrepancy between model and data distributions

$$\text{MMD}^2 = \mathbb{E}_{x, x' \sim p_{\text{data}}} [k(x, x')] - 2\mathbb{E}_{x \sim p_{\text{data}}, y \sim p_{\theta}} [k(x, y)] + \mathbb{E}_{y, y' \sim p_{\theta}} [k(y, y')] \quad (12)$$

- (iii) Stein discrepancy: uses score function, avoids computing normalization

Intuition. born machines can represent distributions that r exponentially hard for classical models. the amplitude of an n -qubit state has 2^n components, so a Born machine over n bits has a “model capacity” thats exponential in n . whether this helps for real-world distributions is an open question.

5.2 Quantum GANs (qGANs)

quantum GANs follow the classical GAN framework but replace one or both networks with quantum circuits:

1. quantum generator, classical discriminator: generator is $U(\theta_G)|0\rangle$, measured samples fed to classical discriminator
2. quantum generator, quantum discriminator: both r quantum circuits, discriminator tries to distinguish real data (loaded into quantum state) from generated samples
3. classical generator, quantum discriminator: less common, but the quantum discriminator might detect patterns the classical one misses

the minimax game:

$$\min_{\theta_G} \max_{\theta_D} \mathbb{E}_{x \sim p_{\text{data}}} [\log D_{\theta_D}(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D_{\theta_D}(G_{\theta_G}(z)))] \quad (13)$$

for the fully quantum version, the discriminator is a quantum circuit that outputs a single qubit measurement:

$$D_{\theta_D}(\rho) = \text{Tr}[M V(\theta_D) \rho V^{\dagger}(\theta_D)] \quad (14)$$

Key Point. qGANs have been demonstrated for loading probability distributions into quantum states (zoufal et al. 2019). the practical use case isn't generating cat pictures — its efficiently preparing quantum states that represent classical probability distributions, which is useful as a subroutine in other quantum algorithms.

6 Quantum Models for Reinforcement Learning

quantum reinforcement learning (QRL) is the least mature subfield of QML, but has some interesting ideas.

6.1 The Setup

in RL, an agent interacts with an environment through a policy $\pi(a|s)$ that maps states to action probabilities. the goal is to maximize expected cumulative reward:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (15)$$

a quantum policy uses a PQC to parameterize π :

$$\pi_\theta(a|s) = |\langle a|U(\theta, s)|0\rangle|^2 \quad (16)$$

where the state s is encoded into the circuit and different measurement outcomes correspond to different actions.

6.2 Quantum Policy Gradient

the REINFORCE algorithm adapts directly:

$$\nabla_\theta J = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t \right] \quad (17)$$

where $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ is the return.

bcs we can use the parameter shift rule to compute $\nabla_\theta \log \pi_\theta$, the quantum policy gradient is well-defined. the circuit acts as a function approximator for the policy, similar to how neural networks are used in classical deep RL.

Example 6.1. quantum policy for CartPole (skolik et al. 2022): 4 state variables (cart position, velocity, pole angle, angular velocity) encoded into a 4-qubit circuit. measurement of first qubit determines action (left/right). with just ~ 20 parameters, the quantum agent matches the performance of a classical neural network with ~ 100 parameters.

Intuition. the potential advantage of quantum RL is parameter efficiency: quantum circuits can represent complex policies with fewer parameters than classical networks. whether this translates to faster learning or better generalization is still debated. the main challenge is that RL already has high variance in gradient estimates, and quantum measurement noise makes it worse.

7 Data Reuploading

this is one of the most important architectural ideas in QML. introduced by pérez-salinas et al. (2020), data reuploading turns a single-qubit circuit into a universal classifier.

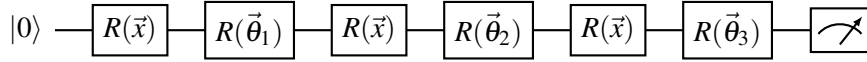
7.1 The Core Idea

instead of encoding data once at the beginning, encode it multiple times throughout the circuit, interleaved with trainable gates:

$$U(\theta, x) = \prod_{l=1}^L \left[W_l(\theta_l) \cdot S(x) \right] \quad (18)$$

each “layer” re-encodes the data and applies a trainable rotation. the key insight: this is analogous to a classical neural network where each layer applies a nonlinear activation to a linear transformation of the input.

7.2 Data Reuploading Circuit

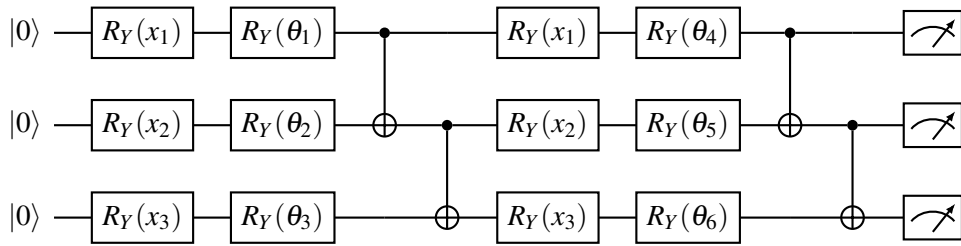


for the single-qubit case, each gate is a general $SU(2)$ rotation:

$$R(\vec{v}) = \exp \left(-i \frac{v_1 X + v_2 Y + v_3 Z}{2} \right) \quad (19)$$

so the encoding gate uses $\vec{v} = \vec{x}$ and the trainable gate uses $\vec{v} = \vec{\theta}_l$.

for multi-qubit data reuploading, add entangling layers between re-uploads:



7.3 Why It Works: Fourier Analysis

the connection to fourier analysis (schuld, sweke, meyer 2021) explains everything. a data reuploading circuit with L layers computes:

$$f(x; \theta) = \sum_{\omega \in \Omega} c_{\omega}(\theta) e^{i\omega x} \quad (20)$$

where Ω is the set of accessible frequencies determined by the encoding gates.

Theorem 7.1 (Universal Approximation via Data Reuploading). a single-qubit data reuploading circuit with L layers can represent any trigonometric polynomial of degree up to L :

$$f(x) = \sum_{k=-L}^L c_k e^{ikx}$$

by choosing appropriate trainable parameters $\theta_1, \dots, \theta_L$. as $L \rightarrow \infty$, this becomes dense in continuous functions on $[0, 2\pi]$ (by standard fourier analysis).

Intuition. each data re-upload adds one more frequency to the fourier spectrum. so L layers of reuploading give u access to frequencies $\{-L, -L+1, \dots, L\}$. the trainable parameters control the fourier coefficients c_k . more layers = higher frequency components = ability to fit more complex functions. this is directly analogous to increasing the width/depth of a classical neural network.

Key Point. data reuploading solved a fundamental limitation of single-encoding circuits. with data encoded only once, a PQC computes a restricted class of functions (limited frequencies). with reuploading, the function class grows with circuit depth, eventually becoming universal. this is wt makes quantum models practical function approximators.

7.4 Practical Considerations

- number of layers L : more layers = more expressive but harder to train (deeper circuit, more parameters, potential barren plateaus)
- encoding gate choice: $R_Z(x)$ gives frequencies $\{-1, 0, 1\}$ per qubit per layer. using $R_Z(x)R_Y(x)$ can give richer frequency spectra
- multi-qubit scaling: with n qubits and L layers, the frequency spectrum grows combinatorially. this is where quantum advantage might come from
- entanglement between re-uploads: CNOT layers between data re-uploads create correlations between qubits, enabling the circuit to capture multi-variate interactions

8 Quantum Transfer Learning

transfer learning is one of the most successful ideas in classical ML: take a model pre-trained on a large dataset, replace the last layer, and fine-tune on ur specific task. the quantum version adapts this by replacing the last layer with a quantum circuit.

8.1 The Architecture

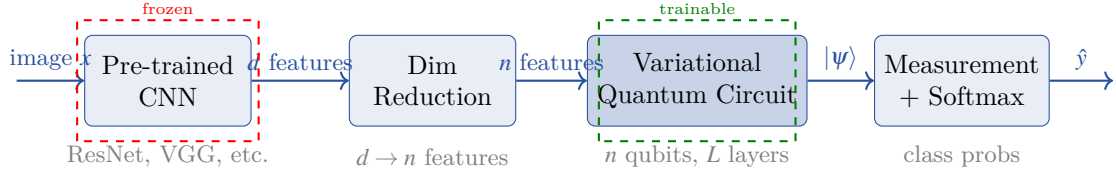
mari et al. (2020) proposed the following pipeline:

1. take a pre-trained classical CNN (e.g., ResNet-18 trained on ImageNet)
2. remove the final classification layer
3. add a dimensionality reduction layer (from d features to n features, where n = number of qubits)
4. feed the reduced features into a variational quantum circuit

5. measure to get class predictions

$$\hat{y} = \text{Measure} \left[U(\theta) \cdot S(\text{Reduce}(\text{CNN}(x))) |0^n\rangle \right] \quad (21)$$

8.2 Architecture Diagram



8.3 Why This Makes Sense

- classical CNNs r great feature extractors: billions of parameters trained on millions of images have learned incredibly useful representations. no quantum model comes close (yet)
- quantum circuits add expressive power at the decision boundary: even a small quantum circuit (4-8 qubits) can implement complex decision functions in the feature space
- few parameters to train: the quantum circuit might have 20-50 parameters, so u need very little data to fine-tune
- practical on current hardware: only need a few qubits, shallow circuits

Example 8.1. transfer learning for medical image classification (mari et al. 2020): ResNet-18 pre-trained on ImageNet, features reduced from 512 to 4, fed into 4-qubit variational circuit. on a binary classification task (ants vs. bees), the hybrid model achieved 96% accuracy, matching the fully classical fine-tuned model. the quantum circuit had only 24 trainable parameters.

Key Point. quantum transfer learning is arguably the most practical QML architecture for real-world image classification right now. it leverages the best of classical ML (feature extraction) and only uses the quantum circuit where it matters (decision making). the downside: its hard to claim quantum advantage when 99% of the work is done classically.

9 Hybrid Classical-Quantum Architectures

there r many ways to combine classical and quantum computation beyond transfer learning. lets categorize them.

9.1 Architecture Patterns

Definition 9.1 (Quantum-Classical Hybrid Architectures). we identify four main patterns:

- quantum layer in classical network: embed a quantum circuit as one layer in a classical deep network. classical layers before and after. this is the “dressed quantum circuit” approach

- (b) classical pre/post-processing: classical networks handle data preprocessing and output post-processing, quantum circuit does the core computation
- (c) parallel quantum-classical: run classical and quantum models on the same input, combine their outputs (ensemble-like)
- (d) alternating quantum-classical: alternate between quantum and classical processing layers, passing data back and forth

9.2 The Dressed Quantum Circuit

the most general hybrid architecture (mari et al. 2020):

$$f(x) = g_{\text{post}}\left(\text{Measure}\left[U(\theta)S(g_{\text{pre}}(x))|0^n\rangle\right]\right) \quad (22)$$

where g_{pre} and g_{post} r classical neural networks.

this gives u the best of both worlds:

- g_{pre} : dimensionality reduction, feature extraction, normalization
- $U(\theta)$: quantum processing in hilbert space
- g_{post} : mapping quantum outputs to desired format (softmax for classification, rescaling for regression, etc.)

the entire pipeline is end-to-end differentiable (using parameter shift rule for the quantum part), so u can train everything jointly with backpropagation.

9.3 Practical Hybrid Training

1. joint training: train all parameters (classical + quantum) simultaneously. requires careful learning rate tuning — quantum parameters often need different learning rates than classical ones
2. alternating training: freeze classical, train quantum, then freeze quantum, train classical. simpler but slower convergence
3. pre-train then fine-tune: pre-train classical parts, then fine-tune quantum circuit. this is the transfer learning approach

Key Point. in practice, the best results come from joint training with separate learning rates: $\eta_{\text{classical}} \sim 10^{-3}$ (Adam) and $\eta_{\text{quantum}} \sim 10^{-1}$ to 10^{-2} (often plain SGD works fine for the quantum part). quantum parameters tend to have smoother loss landscapes and benefit from larger step sizes.

10 Real-World Application Areas

lets survey where QML is being applied (or at least attempted).

10.1 Finance

finance is one of the most active areas for QML research, driven by the potential economic value and the mathematical sophistication of financial models.

portfolio optimization: given n assets with expected returns μ and covariance matrix Σ , the markowitz problem is:

$$\min_w w^T \Sigma w - \lambda \mu^T w \quad \text{s.t.} \quad \sum_i w_i = 1, w_i \geq 0 \quad (23)$$

quantum approaches:

- QAOA for discrete portfolio optimization (selecting k from n assets)
- VQE-based continuous optimization
- quantum annealing (D-Wave) for combinatorial variants

option pricing: pricing european/asian options requires computing expectations under risk-neutral measure:

$$V = e^{-rT} \mathbb{E}^Q[\text{payoff}(S_T)] \quad (24)$$

quantum amplitude estimation can achieve quadratic speedup over classical monte carlo: $O(1/\epsilon)$ vs $O(1/\epsilon^2)$. this is one of the most concrete near-term quantum advantages, though the constant factors are still challenging.

fraud detection: QML classifiers (VQC, QSVC) applied to transaction data. results so far: competitive with classical methods on small datasets but no clear advantage. the bottleneck is data encoding.

10.2 Chemistry and Drug Discovery

molecular simulation: this is arguably the “killer app” for quantum computing, though its more quantum simulation than QML per se. VQE finds ground state energies of molecules:

$$E_0 = \min_{\theta} \langle \psi(\theta) | H | \psi(\theta) \rangle \quad (25)$$

where H is the molecular hamiltonian in second quantization.

drug discovery pipeline: quantum models can contribute at several stages:

- molecular property prediction (QML classifiers/regressors)
- molecular similarity (quantum kernels on molecular fingerprints)
- conformer generation (quantum generative models)
- binding affinity prediction (hybrid quantum-classical models)

Example 10.1. quantum kernel for molecular property prediction (huang et al. 2021): quantum kernels applied to molecular descriptors for predicting solubility. with engineered quantum feature maps that respect molecular symmetries, the quantum kernel matched or slightly exceeded classical kernels (RBF, Tanimoto) on benchmark datasets. the advantage was most pronounced for small training sets (< 100 molecules).

10.3 Optimization

logistics and scheduling: combinatorial optimization problems (vehicle routing, job-shop scheduling, bin packing) mapped to QUBO formulations and solved with QAOA or quantum annealing.

the QUBO formulation:

$$\min_z z^T Q z, \quad z \in \{0, 1\}^n \quad (26)$$

QAOA maps this to a quantum circuit:

$$|\psi(\gamma, \beta)\rangle = \prod_{p=1}^P e^{-i\beta_p H_M} e^{-i\gamma_p H_C} |+\rangle^{\otimes n} \quad (27)$$

where H_C encodes the cost function and $H_M = \sum_i X_i$ is the mixer.

Key Point. tbh, for optimization problems, the evidence for quantum advantage is mixed at best. classical solvers (gurobi, CPLEX) r incredibly well-optimized after decades of development. quantum approaches might eventually win for specific problem structures, but we r not there yet.

10.4 Materials Science

quantum models for predicting material properties:

- band gap prediction using quantum kernels
- phase classification of quantum many-body systems
- materials discovery via quantum generative models

the natural connection: materials r quantum systems, so quantum models might capture relevant physics more naturally than classical ones.

11 Honest Assessment: What Actually Works

time for some real talk. where does QML actually stand?

11.1 What Works (or at Least Shows Promise)

1. quantum kernels for small datasets: when u have < 1000 data points and the quantum kernel captures relevant structure, QSVC can match or beat classical methods. proven advantage exists for specific synthetic problems (liu et al. 2021)
2. hybrid transfer learning: practical on current hardware, competitive results on image classification. but the quantum part contributes marginally
3. quantum simulation / VQE: this is where near-term quantum computers have the clearest path to advantage. not purely ML but uses variational methods
4. generative modeling: born machines can represent distributions that r provably hard for classical models. practical utility still being demonstrated
5. QAOA for specific problem instances: some evidence of advantage for certain combinatorial structures, but not general-purpose

11.2 What Doesn't Work (Yet)

1. QML on big data: current quantum computers have ~ 100 noisy qubits. encoding thousands of high-dimensional data points is impractical. classical ML wins by default on big data
2. replacing deep learning: no quantum model comes close to matching a transformer or modern CNN on standard benchmarks (ImageNet, NLP tasks, etc.). the gap is enormous
3. general-purpose quantum advantage in ML: no one has demonstrated that a quantum ML model outperforms the best classical model on a practically relevant task with real data
4. quantum RL: interesting theoretical framework but limited by noisy hardware and high variance. classical RL algorithms are much more mature

11.3 What Needs More Qubits

1. amplitude encoding of large datasets: encoding N data points into $\log N$ qubits requires coherent operations that scale polynomially — currently infeasible for large N
2. fault-tolerant QML algorithms: quantum PCA, quantum sampling, HHL-based methods all need error-corrected qubits. estimated timeline: 5-15 years for useful problem sizes
3. quantum advantage in kernel methods: the liu et al. result shows advantage exists in principle, but the advantage regime requires circuits that are deeper than current hardware supports with acceptable noise levels

Key Point. the honest summary: QML is a field with beautiful theory, some promising near-term applications (especially in quantum chemistry), but no “smoking gun” practical advantage yet. the most impactful near-term path is probably hybrid classical-quantum approaches that leverage the strengths of both paradigms. if someone tells you they have quantum advantage on a real ML problem with current hardware, be skeptical and ask to see the comparison with properly tuned classical baselines.

12 Looking Forward

12.1 Near-Term Opportunities

- quantum-inspired classical algorithms: sometimes the process of designing quantum algorithms inspires better classical ones (e.g., dequantization results by tang 2019). this is valuable even without a quantum computer
- domain-specific quantum models: instead of trying to beat classical ML generally, design quantum models that exploit domain-specific structure (e.g., molecular symmetries, lattice geometries)
- quantum data: the clearest advantage is for data that comes from quantum systems (quantum sensors, quantum communication, quantum chemistry experiments). here, quantum models have an inherent advantage bcs the data is already quantum
- better classical simulation: understanding quantum ML models often leads to better classical simulation techniques, which benefits everyone

12.2 The Path to Practical Advantage

$$\text{Practical QML Advantage} = \underbrace{\text{Provable Separation}}_{\text{theory}} + \underbrace{\text{Relevant Problem}}_{\text{application}} + \underbrace{\text{Sufficient Hardware}}_{\text{engineering}} \quad (28)$$

we currently have the first ingredient for specific problems, the second is debatable, and the third is years away. but the field is moving fast.

Intuition. dont be a QML doomer and dont be a QML hype person. the technology is real, the math is beautiful, and specific advantages r provable. the path to practical impact requires patience, honest benchmarking, and a willingness to admit when classical methods r simply better. thats how science works.

13 Summary

1. taxonomy: QML splits into kernel methods (convex, limited scale), variational methods (flexible, trainability issues), and fault-tolerant methods (powerful, future hardware)
2. classification: VQC and QSVC r the workhorses. projected kernels help with scaling. no clear advantage over classical methods on standard benchmarks
3. regression: VQR and QKRR extend the classification approaches. bounded outputs from quantum measurements r a feature and a limitation
4. generative models: born machines r quantum-native and can represent hard distributions. qGANs useful for state preparation. training is challenging
5. RL: quantum policy gradients work in principle but face variance and hardware challenges. parameter efficiency is the main potential win
6. data reuploading: re-encode data multiple times to increase expressiveness. fourier analysis explains why it works. universal approximation with enough layers
7. transfer learning: combine classical CNN features with quantum circuits. most practical near-term architecture
8. applications: finance, chemistry, optimization r the main targets. honest assessment: promising but no smoking gun advantage yet

next lecture we tackle the elephant in the room: barren plateaus and trainability. this is wt determines whether variational QML can actually scale, and its one of the most active research areas in the field.